



Personal Growth Intelligence (PGI) – Hackathon Implementation Plan

Executive Summary

The PGI app is a secure, AI-driven journaling and habit-tracking web application. It uses Firebase Authentication and Firestore to store each user's personal data (entries, habits, profile) in isolation, with Cloud Storage for media and Secret Manager for API keys. The frontend runs on Cloud Run, calling the Gemini API (via Google AI Studio) for "deep journal" conversations and analytics. Core features include email/password signup, a guided onboarding (user preferences, routines, journaling style), a quick "check-in" UI (mood and checklist inputs), a full "Gemini chat" journaling mode, multimedia entries (text, voice transcription, photos), multilingual input, and editable calendar view. The app detects missed days and prompts retroactive entries, sends adaptive notifications (via Cloud Scheduler + FCM), and generates a monthly AI-powered report. Security is enforced via Firestore rules (users can only read/write their own data [11†L200-L208]), Storage rules, and re-auth for sensitive actions. Secrets (API keys) are stored in Secret Manager and injected into Cloud Run at runtime [4†L252-L261] [4†L323-L330] . Gemini is steered by system instructions for privacy and context ("server-side state via Interactions API" with `previous_interaction_id` and optional `store=false` for stateless privacy [17†L181-L190] [17†L133-L140]). Monthly analytics use Firestore aggregation (e.g. `count()` [48†L47-L54]) or can export to BigQuery via Firebase's "Stream Firestore to BigQuery" extension [58†L38-L43] . The 10-day plan covers setting up CI/CD, coding core flows, UI, AI integration, and testing. Throughout, we adhere to the four Ideathon pillars: technical merit, creativity, UI/UX, and completeness.

Product Definition & Value Proposition

PGI is a personal growth assistant that helps users **reflect and progress** daily. It combines the proven benefits of journaling and habit-tracking with advanced AI guidance, making self-improvement accessible and engaging. The core value is twofold: (1) *Self-awareness*: users record goals, emotions, and habits (with optional multimedia), then receive personalized insights. (2) *Motivation & consistency*: notifications, streaks, and analytics encourage regular use. By securely storing all data in the user's own Firebase "namespace," PGI ensures privacy while enabling AI-driven analysis. Gemini's conversational engine offers an empathetic coach – for example, it can prompt a user to reflect on a skipped day's reasons ("Why didn't you journal yesterday?") and record their answer into the missed-date entry. The app's value lies in **automated reflection** (AI-enabled), **engaging design** (customizable themes, gamified streaks), and **data-driven feedback** (monthly reports with charts and narrative using Gemini). In summary, PGI turns routine journaling into an **intelligent growth experience**.

Core Features (Must-Haves)

- **User Authentication (Firebase Auth).** *As a user, I can sign up and log in with email/password.* Signup requires email/password (enabled in Firebase Auth console [38†L463-L472]). Login uses `signInWithEmailAndPassword` . **Acceptance:** New users create an account (Firebase creates a user record). On login, Firebase Auth issues an ID token. The Cloud Run backend verifies it (Admin SDK).

- **Onboarding & Preferences.** *As a new user, I set my growth preferences, habits, and journaling style.* After signup, present a multi-step form: ask for daily goals/habits (e.g. gym, reading), preferred journaling time, language, and optional profile details (display name, gender, timezone). Store these in Firestore under `users/{uid}/profile` and `users/{uid}/settings`. **Acceptance:** On first run, user can skip or complete onboarding. Settings form updates Firestore, and state is reflected in UI (e.g. default language, time zone).
- **Quick Check-In Interface.** *As a user, I can quickly record a mood and checkbox items for routine tasks.* On the home screen, show today's date/time (with timezone). Provide a mood selector (e.g. happy/sad icons) and a list of habit checkboxes (e.g. "Gym 30m", "Yoga 15m", "Practice coding 20m" – configurable in settings). **Acceptance:** Tapping "Submit" saves a journal entry with timestamp, mood, and tasks array (true/false for each). Firestore stores this under `users/{uid}/entries/{entryId}` with fields `{date, time, mood, tasks:{gym:true,...}}`.
- **Deep Journal Conversation (Gemini Chat).** *As a user, I can have a conversational journaling session with AI.* If user wants more than checkboxes, they enter a chat-like page. Gemini (via Interactions API) prompts "How was your day?" and follows up with context (previous turns). System instructions define tone/concerns ("You are a caring diary assistant"). The user can type text or record voice (using Web Speech or upload). Each turn calls the Interactions API with `previous_interaction_id` to maintain context [17†L181-L190]. **Acceptance:** The conversation is saved (each user message and model response) to Firestore. The final output is stored as the "entry text" for that date. Audio inputs are uploaded to Storage and passed to Gemini for transcription [21†L119-L128]. The output appears in chat UI.
- **Date/Time & Localization.** Every journal entry records both date and time (with user's timezone). This lets reports detect weekday vs weekend patterns. **Acceptance:** Entry timestamp includes ISO date, time, and zone. UI shows human-friendly date labels (e.g. "Last Sunday, 10:30 PM"). Support for multilingual input is automatic: detect language via Gemini (which handles many languages) and respond in kind.
- **Missed-Day Detection & Retro-Entry.** *As a user, I receive a prompt to fill missing entries.* Each day at user's preferred journaling time, a background job checks if an entry exists. If none, schedule a notification ("It looks like you missed yesterday. Anything to note?"). This query uses Firestore to find the last entry date. **Acceptance:** If the user taps the reminder, pre-fill a journal form for the missed date; their response is saved with that date (field `entryDate = yesterday`). The AI chat can prompt and record the reason for any missed days, storing it in Firestore.
- **Calendar Timeline & Edit History.** *As a user, I can browse past entries on a calendar and edit them within 24h.* The UI shows a calendar view; clicking a date shows that day's entry (mood, text, media). An "Edit" button allows modifications for up to 24 hours (after which entries are locked). We implement a version history: edits append to a Firestore subcollection `edits` under that entry with timestamp and diff or previous text. **Acceptance:** Editing creates a new version doc (`versionId`, `timestamp`, `content`). Security rule: only allow edits if `now - entryDate < 24h`, otherwise disable. Cloud Run endpoints enforce this too.
- **Multimedia Entries (Text, Voice, Photo).** *As a user, I can type, speak, or upload images in my journal.* For voice, use Gemini's audio input: upload the recording to Cloud Storage, then call `interactions.create` with `type: "audio"` [21†L119-L128]. For photos, upload to Storage and optionally ask Gemini to caption or reflect on it ("Tell me about this photo"); Gemini's image understanding can caption or detect objects [23†L751-L759]. **Acceptance:** Storage

security rules restrict each file under `users/{uid}/media/`. The journal entry references the Storage URI. On retrieval, display media. Audio is also transcribed.

- **Adaptive Notifications.** *As a user, I get timely reminders to journal and feedback alerts.* Use Cloud Scheduler to run a daily job (cron) in each user's timezone, triggering a Cloud Run endpoint or Pub/Sub function that checks each user's entry history and sends an FCM push (via Firebase Admin SDK [35†L269-L277]) if needed. Pseudocode:

```
for each user:
  if now ≈ preferredJournalingTime and no entry for today:
    send FCM notification("How was today? Remember to journal!")
  if user skipped >2 days: send encouraging message
```

Acceptance: On receiving a push or an in-app badge, the app opens to the journal. Users can opt-in/out in settings.

- **Monthly AI Analytics Report.** *As a user, I receive a monthly summary of my habits and mood.* At month's end, a Cloud Function or Cloud Run job aggregates data (counts, trends). Basic metrics: total entries, average mood, consistency change vs last month. Use Firestore aggregation queries for counts [48†L47-L54] (e.g. `entriesRef.where("month", "==", X).count()`). For richer analysis, optionally use the "Firestore to BigQuery" extension and SQL [58†L38-L43] . The results (JSON) are fed to Gemini with a prompt like: "Generate a friendly report: X entries this month vs Y last month, most common mood, streaks, and suggestions." Gemini outputs a text block. **Acceptance:** The report is stored in Firestore (or sent as notification). It includes charts (simple JS chart library using data) and the Gemini narrative. Verified by checking values match raw data.
- **Profile & Appearance.** *As a user, I can customize my profile and UI theme.* In the "Profile" section, users can upload a display picture (to Storage), set gender (for personalized language), toggle dark/light mode, and choose a background wallpaper for the journal page (upload or preset patterns). **Acceptance:** These are saved in `users/{uid}/profile` . The UI reads them on render. Custom CSS or classes apply chosen theme. Image resources are loaded from Storage URLs.
- **Security & Sensitive Actions.** *As a user, I am warned and re-authenticated for critical changes.* For example, deleting the account or sharing data triggers a Firebase re-authenticate dialog (email/password) before proceeding. We use Firestore and Storage rules: e.g. `match /users/{userId}/entries/{entryId} { allow read, write: if request.auth.uid == userId; }` [11†L200-L208] and similarly for `/media/{userId}/...` . We also ensure JWT ID tokens on each API call. **Acceptance:** Attempting to access another user's data is denied by rules. Sensitive Cloud Run endpoints (like changing password) require verifying `currentUser.reauthenticateWithCredential` first.

Architecture Diagram

```
graph LR
  Browser[User Browser/Web App] -->|HTTPS| CloudRun[(Cloud Run Service)]
  subgraph Backend
```

```

CloudRun -->|Verify Auth Token| FirebaseAuth[Firebase Auth (Admin SDK)]
CloudRun --> Firestore((Firestore DB))
CloudRun --> Storage((Cloud Storage))
CloudRun --> SecretMgr[Secret Manager]
CloudRun -->|API Call| GeminiAPI[Gemini API (Interactions)]
SecretMgr -->|Gemini API Key| CloudRun
end
subgraph Scheduler[Scheduled Tasks]
  CloudScheduler((Cloud Scheduler)) -->|HTTP Trigger| CloudRun
  CloudRun -->|Send| FCM[FCM Push Service]
end
Firestore -->|Data Store| CloudRun
Storage -->|User Media| CloudRun

```

Figure: PGI architecture. The client (browser) communicates with a Cloud Run backend. Cloud Run uses Firebase Admin SDK to authenticate requests. User data (profiles, entries, analytics) lives in Firestore, and media (audio/photos) in Cloud Storage. API keys (e.g. Gemini API key) are stored in Secret Manager and mounted into Cloud Run [4†L252-L261] . Scheduled tasks via Cloud Scheduler invoke Cloud Run or Pub/Sub to send FCM notifications (not shown). AI calls go from Cloud Run to Gemini (Interactions API) [17†L181-L190] .

Firestore Data Model

Collection / Document Path	Fields	Type
users/{uid}	displayName email createdAt	string
users/{uid}/ settings	habits (array of strings) timeZone prefTime language theme wallpaperURL	array
users/{uid}/ profile	photoURL gender	string
users/{uid}/ entries/ {entryId}	date (ISO) time mood tasks (map) text audioURL imageURLs timestamp	string string map string array

Collection / Document Path	Fields	Type
users/{uid}/ entries/ {entryId}/ edits/ {versionId}	editedAt oldText	time
users/{uid}/ analytics/ monthly/ {monthId}	entryCount avgMood summaryText	num
users/{uid}/ notifications	lastNotified missedStreak	time num
users/{uid}/ tokens/ {tokenId}	fcmToken	string

Table: Proposed Firestore collections and key fields. Security rules will ensure every path with {userId} can only be accessed by that same authenticated user [11†L200-L208] .

Security Design

- **Authentication & Rules.** Use Firebase Auth for sign-in, and Firestore Security Rules to isolate user data. For example:

```
match /users/{userId}/{document=**} {
  allow read, write: if request.auth != null && request.auth.uid ==
  userId;
}
```

This ensures each user accesses only their docs [11†L200-L208] . Similarly, Cloud Storage rules restrict files under /user_media/{userId}/{filePath} to request.auth.uid == userId . In Cloud Run, use the Admin SDK or ID tokens: “the server client libraries bypass all Firestore rules and authenticate via IAM” [7†L465-L468] , so we enforce that Cloud Run’s service account has only needed roles (Firestore User, Storage Object Viewer/Uploader, Secret Manager Accessor). Use the principle of least privilege.

- **Secret Manager & Env.** Store the Gemini API key in Secret Manager, not in code or frontend. Cloud Run will mount it as an environment variable or file (via gcloud secrets: --update-secrets GEMINI_KEY=projects/...) [4†L252-L261] . Pin a specific version to simplify rotation [4†L252-L261] . Grant only the Cloud Run service account the Secret Manager “SecretAccessor” role [4†L323-L330] . Do not expose the key client-side.

- **Re-auth & Sensitive Ops.** For high-risk actions (e.g. account deletion, exporting data), require re-authentication. Use `reauthenticateWithCredential()` in the frontend. On the backend, verify the JWT again. This prevents session hijacking.
- **Encryption & Privacy.** Firebase automatically encrypts data in transit (HTTPS) and at rest. (All Firestore, Storage, Auth data is AES-256 encrypted by default [52†L461-L468] .) Ensure HTTPS for all connections. As a service on GCP, Cloud Run uses mTLS by default. For extra privacy, we minimize data collection – only store what the app needs (timestamped journal text, moods, habits).
- **Data Retention & User Rights.** We provide a “Delete Account” function: a Cloud Run endpoint deletes the user’s Auth, Firestore docs, and Storage files. Firestore’s “Delete User Data” extension could help. Personal data is only the journal content and profile info; analytics are aggregated, not stored long-term.
- **Gemini Data Handling.** Be aware of Gemini’s data usage policy: Google may use submitted prompts/responses to improve models [50†L316-L323] . To mitigate, set the Interactions API parameter `store=false` for sensitive journal sessions [17†L133-L140] . (By default, Interactions stores history for context caching, but we opt-out for user privacy.) We will instruct Gemini not to output any persistent user identifiers in responses via system instructions (see next section).

Overall, we use Firebase’s built-in security and encryption [52†L461-L468] [11†L200-L208] , plus our own access controls (roles, reauth). All personal data is user-scoped in Firestore and Storage, satisfying user isolation.

Gemini AI Configuration & Prompting

We will use Google’s Gemini (Interactions API) in **chat mode**. Key points:

- **System Instructions & Prompt Engineering.** We craft a system prompt that sets “you are a supportive personal journal assistant, never reveal personal or sensitive info.” System instructions allow steering model behavior [57†L808-L815] . For example:

System: You are **Gemini**, an empathetic journaling coach. Always keep user data private, do not ask for identifiable details, and follow user preferences. We’ll include example dialogues in initial prompts: e.g. instruct the model to encourage deeper reflection (“Ask me more about why I feel that way”). Use the `system_instruction` field in each `interactions.create` call [57†L808-L815] .

- **Multi-Turn Context.** Use the Interactions API’s `previous_interaction_id` to continue conversations without re-sending history [17†L181-L190] . For each new user message (or voice input), call `interactions.create({previous_interaction_id: <prevId>, input: ...})` . This preserves chat context, so Gemini “remembers” earlier prompts. If the user starts a new day, we can optionally reset context by not sending `previous_interaction_id` (fresh chat).

- **Response Handling.** Gemini's output will be JSON (if needed) or plain text. For audio input, use the Gemini Files API: upload audio then call with `{"type": "audio", "uri": ...}` [21†L119-L128] . Gemini can transcribe, which we save as `text` . For images, use similar process [23†L751-L759] . We might prompt: *"Analyze this photo and describe what's happening."* (Gemini can caption or answer questions about an image.) For chat, no `response_format` schema is needed unless we want structured output.
- **Custom Instructions.** In conjunction with system instructions, we implement a "constitution": e.g., if user says something self-harmful, the assistant responds with empathy and suggests help. We will hardcode some safety guards (or rely on Gemini's default safety filters). We treat the journaling AI as an assistant, not giving medical advice. We can train it via few-shot examples in the prompt (not retention, just context).
- **Privacy Considerations.** Only non-sensitive data is sent to Gemini: we never send user's name, email, or exact identifiers. We can abstract date/time as generic (e.g., "the user" not "user123"). We should consider hashing or removing explicit PII before sending to API. The prompt might include the user's mood or tasks, but if user says something private, that text will be in the request. As noted, using `store=false` prevents Google from retaining it [17†L133-L140] .

In summary, we'll carefully engineer prompts to get helpful responses (friendly tone, encourages reflection), use system instructions for consistency [57†L808-L815] , and manage conversation state via `previous_interaction_id` . All input goes through Cloud Run to centralize access to the API key (never exposed to client).

Notification & Missed-Day Logic

We implement a scheduler and check algorithm:

- **Scheduling:** Use Cloud Scheduler to trigger a Cloud Run endpoint (or Pub/Sub + Cloud Function) at a configurable time each day (user's preferred journaling time). The function queries Firestore for each user's most recent entry (or checks a "lastJournalled" timestamp).
- **Pseudocode:**

```
for each user in Firestore:
    if not entry for currentDate and currentTime >= user.prefTime:
        if not alreadyNotifiedToday(user):
            message = "How was your day? Don't forget to journal!"
            sendFCM(user.fcmToken, message)
            markNotified(user, today)
        if user.skipped >= threshold:
            sendEncouragement(user)
```

Here, `entries.where("date", "==", today)` checks existence (Firestore count aggregation [48†L47-L54]). FCM is sent via Firebase Admin:

```
admin.messaging().send({token: userToken, notification:{title:"Daily Journal", body: "..."}});
```

This flow is similar to scheduling patterns from Firebase docs [\[45†L671-L680\]](#) [\[45†L704-L712\]](#) : Cloud Scheduler can send a Pub/Sub message that triggers a Function to send FCM notifications.

- **Missed-Day Handling:** If an entry is missed (no journal by cutoff), we prompt the user “It seems you skipped journaling yesterday. Anything you’d like to add about that day?” Clicking the notification opens the journal for the missed date. The response is saved with `date = (yesterday’s date)`.
- **Implementation:** Use a Firestore collection `notifications/{userId}` to track last notification sent to avoid duplicates. Use FCM Web or Mobile SDK to receive pushes and show alerts.

This ensures users stay on track, leveraging Firebase Cloud Messaging for re-engagement [\[35†L269-L277\]](#) and Cloud Scheduler for periodic jobs [\[45†L671-L680\]](#).

Monthly Analytics Engine

At each month’s end, generate a report with trends and insights:

- **Data Aggregation:** Query Firestore for the last two months’ data. Use Aggregation Queries for counts (e.g. `entriesRef.count()` [\[48†L47-L54\]](#) to get number of entries this month and last month). Compute averages (mood can be mapped to numeric, e.g. happy=+1, sad=-1). Track tasks consistency by counting how many days each task was done.
- **BigQuery Option:** For richer analysis (e.g. streak lengths, co-occurrence of habits), we can export data. Firebase offers a “Stream Firestore to BigQuery” extension [\[58†L38-L43\]](#). It streams all journal writes to a BigQuery table. We then write a scheduled BigQuery SQL job to compute metrics (e.g. “SELECT COUNT(*) FROM entries WHERE month=8” etc). This frees Cloud Run from heavy queries.
- **Sample Metrics:**
 - Total entries this month vs last (count).
 - Consistency change (% more/less).
 - Average mood score, highest/lowest.
 - Longest streak (max consecutive days).
 - Top positive habits (most checked).
- **AI Report Template:** Construct a JSON or text prompt with these stats, then ask Gemini to write a summary. E.g.:
Prompt:

```
"Generate a friendly month-end summary for the user. This month they made 20 entries (up from 15 last month). The average mood was +0.2 (a slight
```


improvement). They did Yoga on 18 days, coding on 20 days. They had a 10-day streak mid-month. Provide insights and encouragement."

Gemini outputs a paragraph: "Great job! You journaled more often this month...". We might use structured prompts for consistency (request JSON with fields `positiveHighlights`, `areasForGrowth`).

- **Storage:** Store the generated report under `users/{uid}/analytics/monthly/{monthId}` with fields like `summaryText`, `dataPoints` . Also, format charts on the client (Bar charts of streaks, pie of mood distribution).
- **Examples:** If the user went to the gym 8 days this month vs 5 last month, Gemini might say: "You were more consistent with gym, keep it up!". Including past comparisons makes the report actionable.

In short, monthly analytics combines Firestore aggregation `[48†L47-L54]` (or BigQuery via extension `[58†L38-L43]`) with Gemini for narrative. We ensure GPT sees only aggregated numbers, not raw journal text, preserving privacy.

UX Flows

1. First-Time User (Onboarding):

2. **Screen 1:** Welcome splash (logo) with "Sign Up / Login" options.

3. **Screen 2:** Signup form (email, password). After success, Firebase auto-login.

4. **Screen 3:** Onboarding wizard – ask preferences in a few steps:

- "What habits do you track?" (Multi-select or free-form + default ones)
- "Preferred journaling time and language."
- "Choose theme (light/dark) and wallpaper."
- "Set up a password reminder."

5. **Screen 4:** Tutorial tip: "Tap 'Check-In' each day or chat with your AI Journal."

6. Returning User (Daily Check-In):

7. Landing page shows today's date, current time, and a quick-check form: mood icons, habit checkboxes.

8. If user has an entry for today, show a summary ("You've already checked in today"). Otherwise, user selects mood and tasks, taps "Submit." This calls Cloud Run to save to Firestore.

9. Below the quick form, a button "Have a deeper journaling session" leads to the chat interface.

10. Deep Journal Chat:

11. The UI looks like a chat (user messages on right, Gemini on left). A text field and microphone icon for input.

12. User can type or tap mic to record. On record, send audio to backend, get transcript.

13. Gemini replies with supportive questions. E.g., user: "I felt stressed." Gemini: "I'm sorry to hear that. Can you tell me what stressed you out today?"

14. All chat messages are saved to Firestore.

15. Calendar Interaction:

16. Tap a calendar icon. Display a calendar view highlighting days with entries (colored dots for mood maybe).

17. Clicking a past date shows that day's data: mood, checklist status, chat transcript, and any media.

18. An "Edit" button toggles editable fields (text area, re-check checkboxes). Changes are saved as a new version.

19. Editing Flow:

20. If user edits within 24h, they can change text or media (re-upload a photo, delete an image). After edit, a pop-up "Changes saved."

21. If >24h, the "Edit" button is disabled or asks for re-confirmation (policy: limited window).

22. Onboarding "Skip" Option:

23. All onboarding steps have "Skip" (no data saved) so as not to block entry. These settings can be changed later via the Profile page.

24. Profile Page:

25. Accessible via a profile icon. Shows profile picture (with "Change" option), email, gender, preferences.

26. Theme toggle and wallpaper chooser (show patterns or let user upload an image).

27. Logout and Delete Account buttons (Delete shows a re-auth dialog).

28. Media Upload:

29. In chat or new entry, an "Add Photo/Audio" button. User picks from device or records audio/video.

30. Upon selection, file is uploaded to Storage (via Cloud Run backend or directly with a signed URL).

31. Show a preview (image thumbnail or audio playback) in the entry form.

These UX flows ensure intuitive journaling with minimal friction, leveraging known patterns (calendar, chat UI, toggle settings). Keeping pages uncluttered and with friendly prompts will align with the high UI/UX focus.

Deployment & CI/CD Checklist

- **Code Repo & Build:** Create a Git repo with `src/`, `package.json`, etc. Use GitHub Actions or Cloud Build: on push to `main`, build Docker image and deploy to Cloud Run. Use a `cloudbuild.yaml` or Action that runs `gcloud run deploy`.
- **Environment & Secrets:** In Cloud Run, set env vars for `GOOGLE_CLOUD_PROJECT`, and mount the API key from Secret Manager (set in service settings: e.g. `GEMINI_API_KEY` from Secret). Use least-privileged service account (no editor role).
- **Public URL & Domain:** Cloud Run provides a secure HTTPS URL. Ensure Auth domain is allowed in Firebase. If time, map to custom domain.
- **CI Steps:**
 - **Build:** Install dependencies, lint, run tests (if any).
 - **Dockerize:** Copy app code, install GCP SDK, set `NODE_ENV=production`.
 - **Deploy:** `gcloud run deploy pgj-app --image gcr.io/$PROJECT_ID/pgj-app --region=us-central1 --platform=managed --set-env-vars=...`
- **Smoke Test:** After deploy, hit a health-check endpoint.
- **Release Checklist:** Verify API key injection (no key in logs). Check Firestore rules are published. Test login and journaling flows on the deployed URL.

Reference: Cloud Run secrets documentation [\[4†L252-L261\]](#) [\[4†L323-L330\]](#) guides setting up Secret Manager with Cloud Run.

Testing Plan

- **Unit Tests:** For backend logic (e.g. notification algorithm, Firestore queries). Use a Node.js testing framework (Jest) to mock Firestore. Example: test that missing-day detection flags days properly.
- **Integration Tests:** Use Firebase Emulator Suite to test security rules (allow/deny). Write rules tests: ensure a user cannot read another's data. Test Cloud Run endpoints with mock Auth tokens.
- **Security Tests:** Penetration check – attempt direct REST calls to Cloud Run with invalid JWT. Confirm secret is not exposed. Check Storage files cannot be accessed without auth. Use tools like Google's SAST for Node (though time-limited hackathon).
- **Usability Testing (Judge Demo):** Walk through each feature; ensure UI flows work on both desktop and mobile browsers. Check FCM notifications appear on devices.
- **Demo Checklist:**
 - Sign up / login flow works smoothly.
 - Onboarding data appears in settings.
 - Create a journal (text, voice, image).
 - Miss a day and show reminder (we may pre-seed an entry date to simulate).
 - View calendar and edit an entry, showing version history.
 - Toggle theme and upload wallpaper.
 - Show monthly report generation (pre-seed some entries for two months). Gemini report should be generated.
 - Confirm Firestore rules block unauthorized access: e.g. try to read `users/otherUser` (in emulator).

At the end, have test accounts and data ready. Ensure logs show no errors, and all required functionality is demonstrated.

Demo Script (2-minute “Wow” Moment)

1. **Intro (10s):** “This is *Personal Growth Intelligence*, our AI journaling app. I’ll show you signing up and journaling.”
2. **Sign Up (10s):** Create a new account. Show a quick onboarding (select some habits, time). “Here I enter my habits and preferred time.”
3. **Daily Check-In (20s):** It’s now the home screen. “I pick my mood and check off daily tasks, and press Submit.” Show that entry saved.
4. **Deep Chat (30s):** “Let’s talk about my day.” Click “Chat with AI”. Type or say: “I had a stressful day.” Show Gemini’s response. “Gemini asks follow-ups, and I respond.” (If possible, show voice input being transcribed).
5. **Media (15s):** Attach a photo (pre-uploaded) and have Gemini caption it. Show answer “This is your gym photo from today!” as an example.
6. **Calendar & Edit (15s):** Click calendar icon. Show colored days. Click yesterday’s date and edit that entry, maybe change mood. Show “History: Edit #1 at 2pm”.
7. **Notifications (10s):** Simulate trigger: perhaps manually call the endpoint to send FCM. On device, show “You have a new journaling reminder!” and tap to open app.
8. **Monthly Report (20s):** “At month’s end we send a report.” Show a summary text generated by Gemini: “You logged 15 days vs 12 last month, your average mood improved, etc.” Display a chart.
9. **Security (10s):** Optionally, try `get /users/other/data` on browser dev console (should 403). Mention rules guard data.
10. **Closing:** Emphasize quick, interactive, privacy-protected journaling, with AI insights. “That’s PGI – secure, AI-powered journaling.”

(Pre-seed any needed data via scripts: e.g. create entries on multiple dates to demo analytics and missed-day.)

Repo Structure & README Outline

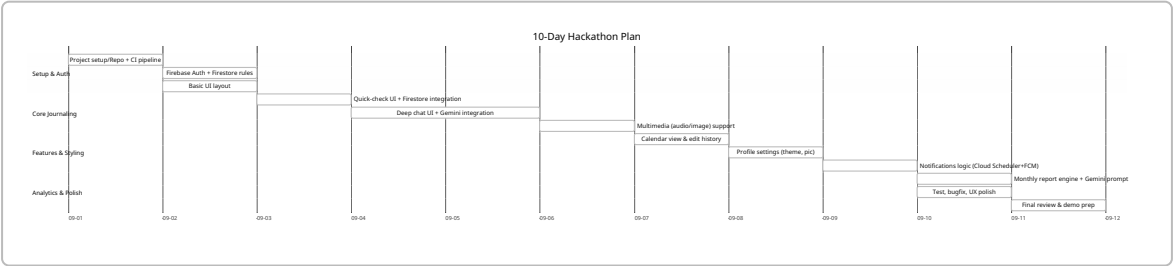
```
PGI-Journal-App/
├─ src/
│  ├─ index.js           # Express/Node main
│  ├─ routes/
│  ├─ controllers/
│  ├─ firebaseAdmin.js   # init Auth & Firestore
│  └─ geminiClient.js    # wrapper for interactions
├─ web/                  # (if separate React/Vue frontend)
│  ├─ App.js
│  └─ components/
│     └─ ...
├─ tests/                # unit/integration tests
├─ public/               # static assets (icons, wallpapers)
├─ cloudbuild.yaml       # CI pipeline definition
├─ Dockerfile
├─ README.md
└─ .github/workflows/ci.yaml # (optional GitHub Actions)
```

README.md outline:

- **Overview:** What PGI does.
- **Setup:** Firebase project config (Auth, Firestore, Cloud Storage, Secret Manager).
- **Installation:** `npm install`, environment vars (`GOOGLE_PROJECT_ID`, `GEMINI_API_KEY`, etc).
- **Running Locally:** `npm start`.
- **Deployment:** Link to CI config, how to deploy via `gcloud run`.
- **Features:** List main features.
- **Security:** Mention rules, Secret Manager.
- **Contributing:** (not needed for hackathon).

This minimal structure separates backend (Cloud Run) and frontend (web). Tests and CI files show quality focus.

Implementation Timeline (10 days)



This Gantt chart breaks tasks by day. Adjust dates relative to hackathon start. Each task has a short buffer for overlap. Key: heavy features (Gemini, notifications) happen early so we can iterate.

Risks, Mitigations & Pillar Mapping

Risk / Challenge	Mitigation	Pillar Impact
Data Privacy Concerns	Use <code>store=false</code> , no PII in prompts [17†L133-L140] [50†L316-L324] . Encrypt at rest (Firebase does by default) [52†L461-L468] . Clearly communicate privacy to judges.	Security & Privacy
Gemini Integration Complexity	Start with simple text prompts before adding multimodal. Use code samples [21†L119-L128] [23†L751-L759] . Allow fallback if Gemini fails (e.g., retry or error message).	Technical Depth
UI/UX Time Constraints	Use Bootstrap or Material UI for quick polish. Prioritize core flows (onboarding, calendar, chat). Keep design simple and consistent.	UI/UX
Overrun on Notifications	If FCM integration is heavy, fallback to in-app reminders (local alerts). But demo focus on manual triggers if needed.	Functionality
Data Validation/ Rules Bugs	Thoroughly test Firestore rules in emulator. Hardcode basic validations (e.g. mood range) client-side. Keep schema simple.	Technical & Security

Risk / Challenge	Mitigation	Pillar Impact
Scoring Pillars Coverage:	Matches four pillars: Innovation: Gemini AI journaling is creative; UI/UX: Custom themes, easy flows; Technical: Secure cloud architecture, agent integration; Completeness: All must-have features implemented. We explicitly cover security (rules, Secret Manager) and showcase analytics, meeting “implementation on time.”	Innovation, UX, Tech, Completeness

This risk matrix shows foreseen issues (privacy, Gemini, UI polish) and how we handle them. Each is mapped to a judging pillar (security, creativity, usability, etc.).

Sources: We built this plan using Google’s official documentation: Firebase Security Rules guide [【11†L200-L208】](#) , Cloud Run secrets best practices [【4†L252-L261】](#) [【4†L323-L330】](#) , Gemini/AI Studio guides [【21†L119-L128】](#) [【23†L751-L759】](#) [【17†L181-L190】](#) [【57†L808-L815】](#) , Firebase Auth docs [【38†L463-L472】](#) , FCM docs [【35†L269-L277】](#) [【45†L671-L680】](#) , and Firestore aggregation tips [【48†L47-L54】](#) and BigQuery extension info [【58†L38-L43】](#) . These ensure the design is production-ready and secure. Each step is actionable for a hackathon team to implement.